

Predicting Passage of Nebraska Legislative Bills

Capstone for HarvardX Data Science Professional Certificate

Sara E. Brady, Ph.D.

2026-06-09

Contents

Executive Summary	2
Introduction	2
Methodology	3
Training and Testing Data Sets	10
Preprocessing	11
Training	16
Results	28
Discussion	30
Conclusion	31
References	32

Executive Summary

During its recent 109th legislative session, the Nebraska Legislature introduced 1,262 bills. Of those bills, only 361 (28.6%) became state laws (Nebraska Legislature, 2026b). As legislative bills are introduced and move their way through the lawmaking process, it may be useful to know which bills have a higher chance of passing. A bill's characteristics could include if a bill was carried over from a previous legislative session or whether the bill is a priority bill.

The purpose of this project was to collect data on legislative bills introduced from 2007 through 2024 and evaluate several machine learning algorithms in predicting the passage of a Nebraska legislative bill. For this project, the outcome variable was whether or not a bill was passed into law, which included bills that passed, as well as bills that were amended into package bills.

Information on 11,584 legislative bills introduced in the Nebraska Legislature from 2007 to 2024 were webscraped from the Nebraska Legislature's website. Using a training data set of 7,372 bills, model evaluation was conducted via repeated 10-fold cross-validation across three iterations for five commonly used classification models: generalized linear model, LASSO regression, random forest, decision tree, and k-nearest neighbor. After comparing the predictive accuracy of the models on a test data ($N = 2,457$), the generalized linear model and LASSO regression demonstrated the highest accuracy.

Since there were no significant differences between these two model, the simpler generalized linear model was selected for testing on an independent test data set of 1,587 bills. The final generalized linear model correctly predicted outcomes for 90.4% of bills with exceptionally high discrimination in predicting the outcome of a bill. Based on variable importance metrics, the strongest predictors included whether the bill was a package bill (or amended into a package bill), was placed on General File, was not a carryover bill, and was introduced in the first regular session.

This project has the potential to impact the lawmaking process in Nebraska and provide policymakers, advocates, and communities with basic indicators on how to strategize legislative priorities. This project could be improved by including the political affiliation of bill introducers, as well as identifying novel approaches for gathering important bill information such as fiscal impact. In the future, policy researchers should consider adding future legislative bills to this data set so that training data may be expanded and refined.

Introduction

Nebraska Unicameral

Nebraska is the only state in the United States whose state government does not include two legislative chambers. As a unicameral legislature, the Nebraska Legislature consists of 49 senators, each representing legislative districts containing about 40,000 citizens (Nebraska Legislature, n.d.). Convening at the start of January of each year, the Nebraska Legislature governs within a biennium, which is a two-year cycle whereby the first session meets for 90 days in odd-numbered years and the second session meets for 60 days in even-numbered years. The Governor may also convene the Nebraska Legislature for a special session, which occurs during the interim period in the summer or fall.

Process of Lawmaking

All legislative bills must be introduced within the first 10 days of either the first or second session of each biennium (Nebraska Legislature, 2026a). Once a bill is introduced, it is referred to a committee where all bills must have a public hearing. If a committee decides to advance a bill, it will be placed on General File, whereby the Speaker will decide to place the bill on the agenda for the first round of debate. From there, a bill can be voted to advance to Select File (second round of debate), and then on to Final Reading (third round of debate).

If the Legislature passes the bill at Final Reading, the bill is sent to the Governor who can either veto or sign the bill into law. Occasionally, if the Governor declines to sign a bill, the bill automatically becomes state law. Legislative bills introduced within the first regular session that do not become law are carried over to the second legislative session within the biennium. If these carryover bills are not signed into law before the conclusion of the second legislative session, they are indefinitely postponed and must be introduced again at the beginning of the next biennium, thus, starting the process all over again.

Legislative Journal Chronology

Historical information on all legislative bills introduced in the Nebraska Legislature are available on its website, nebraskalegislature.gov. To date, the most recent journal chronology posted to the Legislature’s website is from 2025, which was the first regular session of the 109th Legislature (Nebraska Legislature, 2025). Researchers can use these archived documents for determining the history of a legislative bill, including but not limited to date of introduction, who introduced the bill (i.e., senator, speaker, or committee), when the bill was placed on General File, Select File, or Final Reading, when the bill was signed into law or returned without the governor’s approval (i.e., vetoed).

Current Project

Given that Nebraska’s journal chronologies provide substantive information about the history of all legislative bills introduced within a session, the purpose of the current project was to (1) create a data set that summarizes Nebraska legislative bill information over a certain time frame and (2) use that data set to predict the passing rates of bills introduced during a legislative session. To build the data set, text scraping methods were used to scrape journal chronology information from PDF documents posted to the Nebraska Legislature’s website. The final data set included 11,584 legislative bills and was built from the journal chronologies of 2007 (100th Legislature) through 2024 (108th Legislature). To predict Nebraska bill passing rates, machine learning algorithms were trained, tested, and evaluated for accuracy and interpretability. To prevent overfitting the data, the full data set was first partitioned into a larger training set and a holdout test set. The larger training set encompassed legislative bills introduced between 2007 and 2022 ($n = 9,833$). At the time of this writing, the full journal chronology of the 109th Legislature was not available. Therefore, the final holdout test set included legislative bills introduced during the 108th Legislature (2023-2024; $n = 1,587$). Using other cross-validation techniques, the larger training set was used for determining the final model. Once the final model was chosen, the chosen machine learning algorithm was tested on the holdout test set.

Methodology

Data Collection

Using R’s `pdftools` package, the full data set ($N = 11,584$) was scraped from the PDFs of journal chronologies that spanned the years 2007 to 2024. Journal chronologies are provided annually and designated as either being part of a regular or special session. For each legislative session, there are two regular sessions. If a special session was convened, those journal chronologies were also scraped. From the journal chronologies, the following variables were collected for each bill (see Table 1).

Table 1: Legislative Bill Variables Collected from Journal Chronologies

Variable	Type	Description
Legislature	String	Which legislative session the bill was introduced in
Special Session	Logical	If bill was introduced during a special session
Bill Number	Integer	Unique identifier of the bill

Table 1: Legislative Bill Variables Collected from Journal Chronologies (*continued*)

Variable	Type	Description
Introducer	String	Senator or committee that introduced the bill
Date of Introduction	String	Date bill was introduced
Amended Into Other Bill	Logical	If provisions of bill was amended into another bill
Committee	String	Committee the bill was referred or rereferred to
Bill Withdrawn	Logical	If bill was withdrawn
Carryover Bill	Logical	If bill was a carryover bill from the first regular session of biennium
Priority Bill	Logical	If bill is a priority bill
Priority Owner	String	If bill is a priority bill, which committee or senator declared it as a priority bill
A Bill	Logical	If bill is categorized as an appropriation bill (A Bill)
Has A Bill	Logical	If bill has a corresponding A Bill
Placed on General File	Logical	If bill was placed on General File
Enrollment and Review Initial	Logical	If bill was advanced for Enrollment and Review Initial
Placed on Select File	Logical	If bill was placed on Select File
Enrollment and Review	Logical	If bill was advanced for Enrollment and Review
Engrossment		Engrossment
Placed on Final Reading	Logical	If bill was placed on Final Reading
Passed Over	Logical	If bill was passed over during debate
Failed on Final Reading	Logical	If bill failed to advance on Final Reading
Approved by Governor	Logical	If bill was approved by the governor
Became Law Without Governor	Logical	If bill became law without the governor’s signature
Vetoed	Logical	If bill was returned to the Legislature without the governor’s approval
Veto Override	Logical	If bill was passed by the Legislature notwithstanding objections of the governor
Indefinitely Postponed	Logical	If bill was indefinitely postponed

During and after the scraping process, validation checks were made to confirm the accuracy of the data. If errors during the scraping process were detected, updates to the pdf scraping functions were made. Errors detected in the journal chronologies were verified against the full journal records listed on the Legislature’s website. After all legislative sessions were merged into one data frame, several additional variables were calculated (see Table 2).

Two variables, introducer type and term limit, were calculated based upon senator characteristics that were merged from a separate data frame. Because senator rosters are not archived for each legislative session separately, senator information was scraped from legislative update archives posted to the Nebraska’s Legislature’s official news site, *Unicameral Update* (Unicameral Update, 2026). PDFs of archives dating back to 2004 were scraped for all current senator’s last names, as well as the speaker for each legislative session. Unique name identifiers were assigned to all senators to accommodate senators with the same last names and senators who changed their names between sessions. To determine if a senator was term limited in a given year, the number of consecutive years that they appeared in the data set was calculated. State senators can only serve for two consecutive legislative sessions (i.e., four consecutive years). Therefore, term-limited senators were identified if they appeared in the data set for four consecutive years. Only state senators who were appointed to replace a former senator appeared in the data set for five consecutive years. To verify that these five consecutive years were not data collection errors, senator appointments were confirmed through their public profiles on ballotpedia.org.

Table 2: Calculated Variables from Journal Chronology Data

Calculated Variable	Type	Description
Passed	Logical	If bill was approved by the governor, became law without the governor's signature, or was passed by the Legislature after being vetoed by the governor
Passed (All Bills)	Logical	If a bill or provisions of a bill amended into a package bill passed
Number of Bills Amended	Integer	The number of other bills that were amended into a bill
Package Bill	Logical	If bill is a package bill (i.e., a bill has at least one other bill amended into it)
Package Bill Category	String	Whether a bill is a package bill (multiple bills amended into it), amended into a package bill (provisions of bill amended into another bill, also called a "rider" bill), or a standalone bill (not a package bill or a rider bill)
Introducer Type	String	Whether bill was introduced by a senator, committee, or speaker
Term Limit	Logical	If bill was introduced by a term-limited senator

Final data set

The final data set included 11,584 legislative bills and appropriation (A) bills introduced between 2007 and 2024 and included 47 variables. To visualize the the flow of legislative bills once they were introduced, a Sankey plot was created using the networkD3 package in R (see Figure 1). As shown in Figure 1, only 30.1% of bills passed (i.e., became law). Although 64.5% of bills placed on General File passed, the passing rates for bills placed on Select File (93.8%) and Final Reading (97.7%) were much higher. This suggests that once a bill advances from General File it is highly likely to pass. The Sankey plot does not visualize the number of bills that were amended into bills that became law. For the purpose of this analysis, the main outcome variable is the *passing rate of all bills, including bills amended into other bills*. To determine which variables were most correlated with a bill passing, further exploratory data analysis was conducted.

Bill Outcome per Session

When examining bill outcome by legislative session, the number of total bills introduced increased but the pass rate decreased over time (see Figure 2). For example, during the 2007-2008 legislative session 31.5% of 1,268 bills passed. However, during 2023-2024 only 15.9% of 1,606 bills passed. This trend is likely due to the fact that more bills were being amended into package bills over time.

A package bill is a bill that contains multiple "rider" bills. Package bills (also sometimes called a "Christmas tree" bill) are constructed for expediency when bills amended into package bills (rider bills) pertain to a related topic. If a package bill passes, only the package bill's legislative bill number is signed into law. A rider bill's number is, then, indefinitely postponed. However, for policymakers and advocates of rider bills, the passage of a package bill is considered a legislative win for their particular rider bill. Although rider bills are indefinitely postponed, some or all of their provisions are passed into law through package bills. Therefore, expanding the definition of a bill passing should also include rider bills.

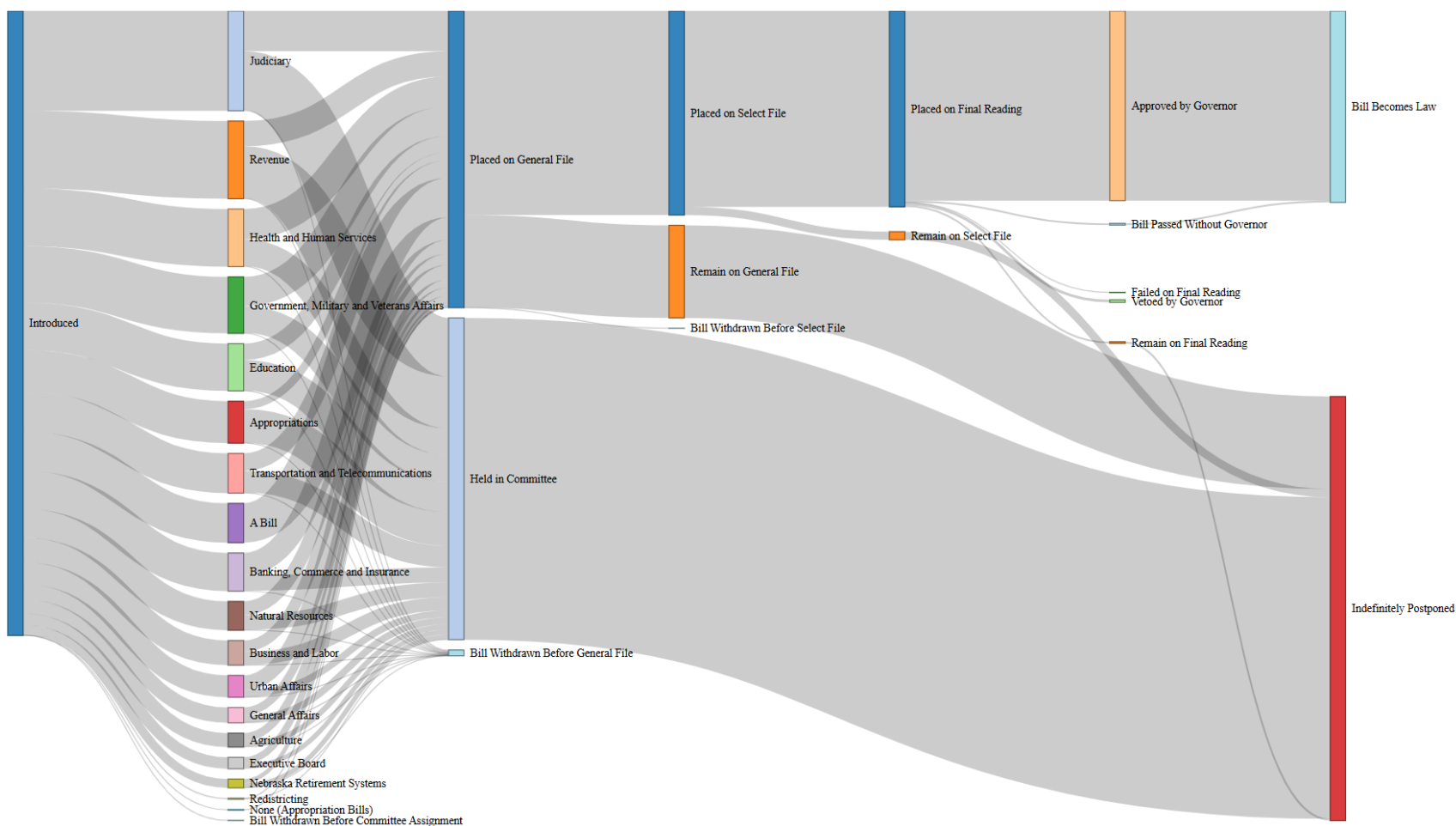


Figure 1: Pathway of All Bills Introduced in the Nebraska Legislature from 2007 to 2024 ($N = 11,584$)

Note. All introduced bills are referred to a committee with the exception of appropriation (A) bills and some special session bills. Once a bill is voted out of committee, it is placed on General File (the first round of debate), where it can be voted to advance to Select File (the second round of debate). If a bill is voted to advance to Final Reading (the third round of debate), it can either fail on final reading or be voted to be presented to the Governor. If the Governor approves the bill, the bill becomes law. Rarely, the Governor can veto a bill and, then, the Legislature overrules the veto and the bill becomes law. Or, the bill can remain unsigned by the Governor and, then, it automatically becomes law.

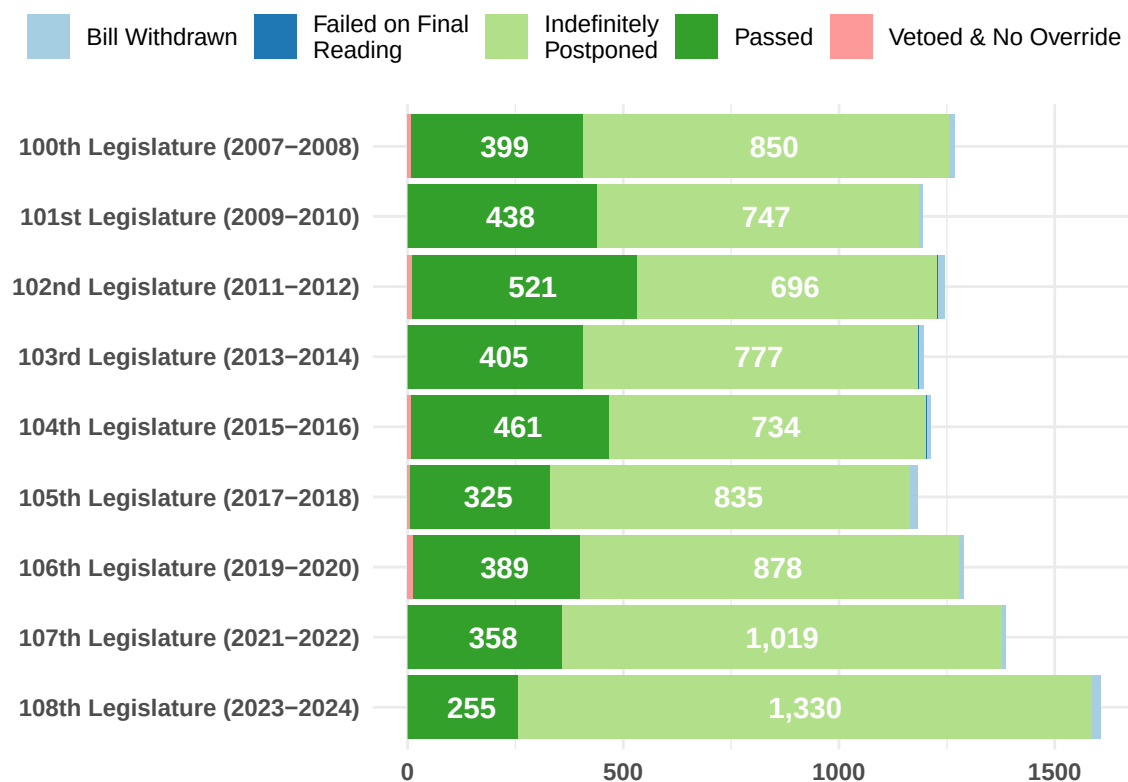


Figure 2: Number of Bills per Legislative Session by Bill Outcome

Pass Rates for All Bills

When expanding the definition of passing to include standalone bills, package bills, and rider bills, the passing rates ranged from 39-51% and remained relatively similar over time. Standalone bills, moreover, had a much lower passing rate (33%) than package bills (97%) or bills that were amended into package bills (98%; see Table 3).

Table 3: Pass Rates for All Bills (Including Bills Amended Into Package Bills)

Variable	Percentage of Passed Bills
Legislative Years	
100th Legislature (2007-2008)	44
101st Legislature (2009-2010)	49
102nd Legislature (2011-2012)	50
103rd Legislature (2013-2014)	48
104th Legislature (2015-2016)	51
105th Legislature (2017-2018)	39
106th Legislature (2019-2020)	47
107th Legislature (2021-2022)	43
108th Legislature (2023-2024)	42
Introduced Biennium	
First	47
Second	45
Special	21

Table 3: Pass Rates for All Bills (Including Bills Amended Into Package Bills) (*continued*)

Variable	Percentage of Passed Bills
Introducer Category	
Committee	81
Senator	44
Speaker	75
Amended Into Package Bill	
Was Amended Into Another Bill	98
Was Not Amended into Another Bill	36
Package Bill	
Is Not a Package Bill	43
Is a Package Bill	97
Package Bill Category	
Amended Into Package Bill	98
Package Bill	97
Standalone Bill	33
Carryover Bill	
Is Carryover Bill	22
Is Not Carryover Bill	58
Priority Bill	
Is Not a Priority Bill	40
Is a Priority Bill	79
Priority Owner	
Speaker	88
Committee	85
Senator	71
Is Not a Priority Bill	40
Introduced by Term-Limited Senator	
Introduced by Term-Limited Senator	47
Was Not Introduced by Term-Limited Senator	46
A-Bill	
Is Not an A-Bill	44
Is an A-Bill	69
Has A-Bill	
Does Not Have an A-Bill	41
Has a Corresponding A-Bill	76
Amended-Into Bill (Provisions) Has an A Bill	
Amended-Into Bill Has an A-Bill	99
Was Not Amended Into Another Bill or Amended-Into Has no A-Bill	42
Amended-Into Bill (Provisions) was Passed	
Amended-Into Bill was Passed	100
Was Not Amended Into Another Bill or Amended-Into Bill was not Passed	36

Continuous variables that were included in the main data set include the legislative session (i.e., 100, 101, etc.), the number of names added to a bill, and the number of bills amended into a package bill. Because there were a sizable proportion of bills with zero names added, Figure 3 shows the distribution of the number of names added per bill for only bills with at least one name added by legislative session. Figure 4 shows the distribution of the number bills amended into package bills per legislative session.

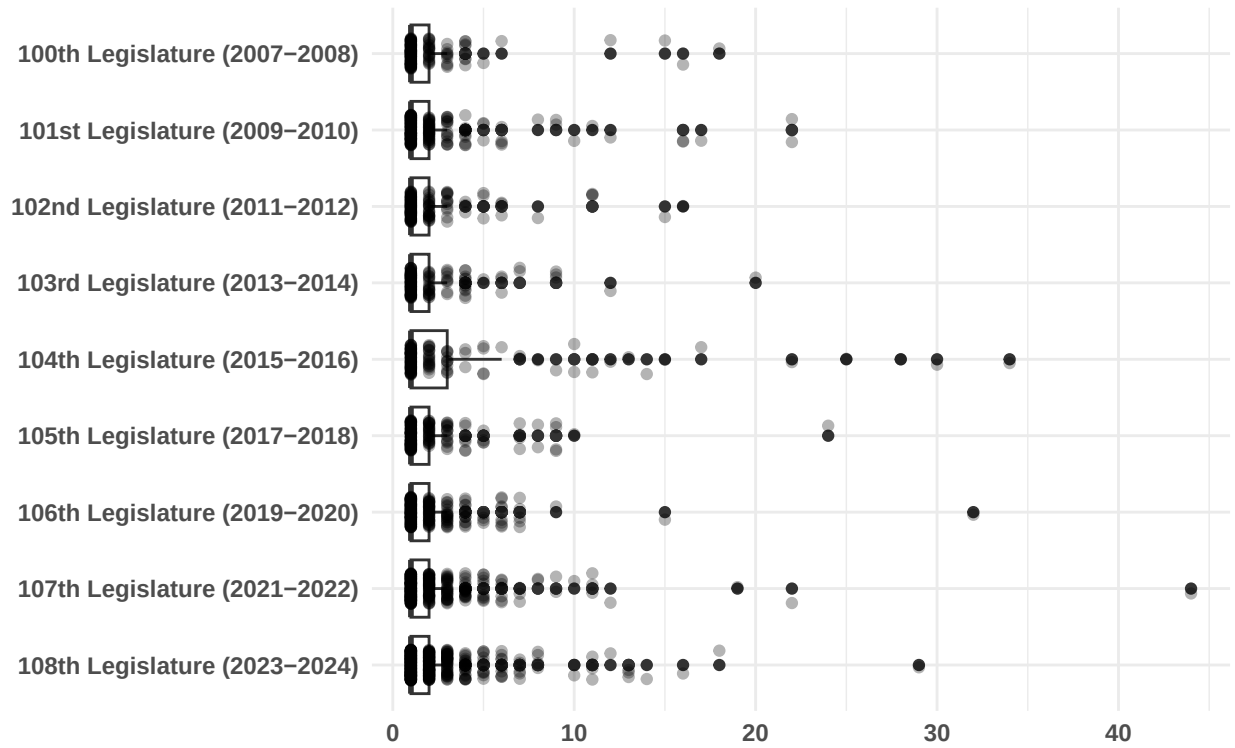


Figure 3: Frequency of Names Added to Introduced Bills with at Least One Name Added

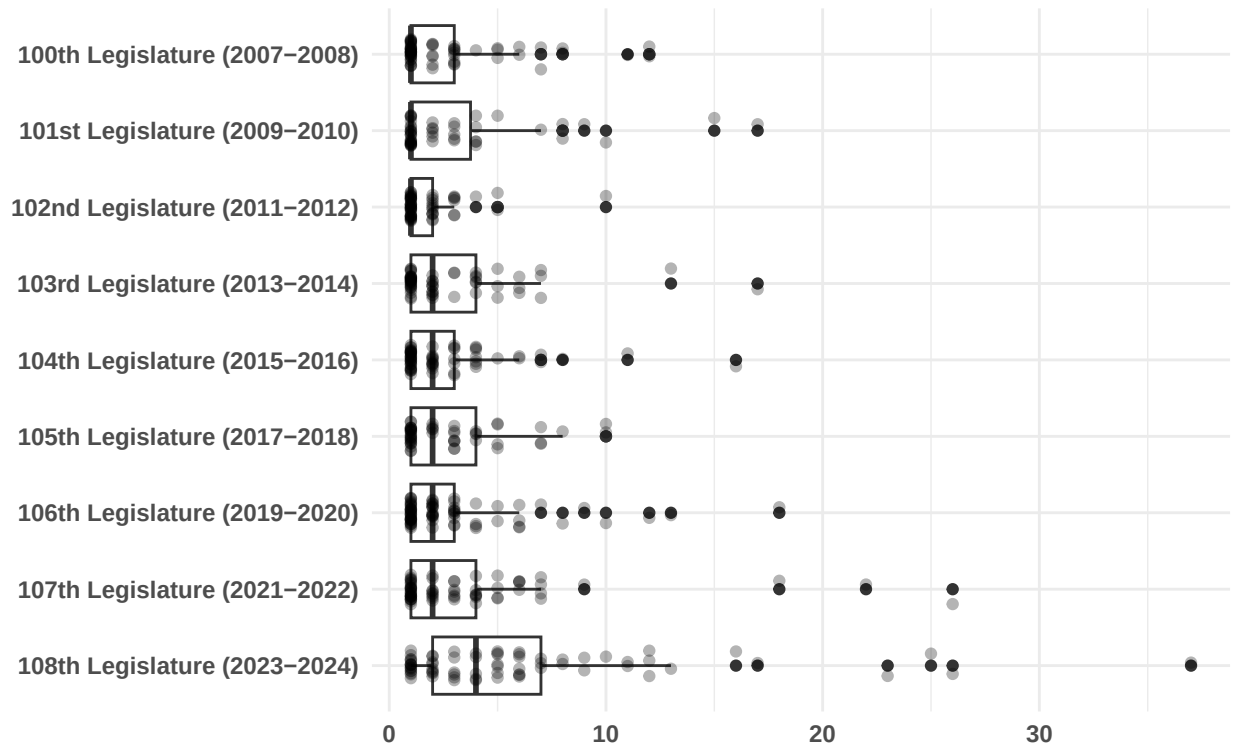


Figure 4: Number of Bills Amended Into Package Bills

The distribution of bills that passed vs. bills that did not pass was similar for the number of names added to a bill (Figure 5), as well as the number of bills amended into package bills (Figure 6). Given that the number of bills amended into package bills only applies to package bills, this variable was not considered for inclusion in the main training data set.

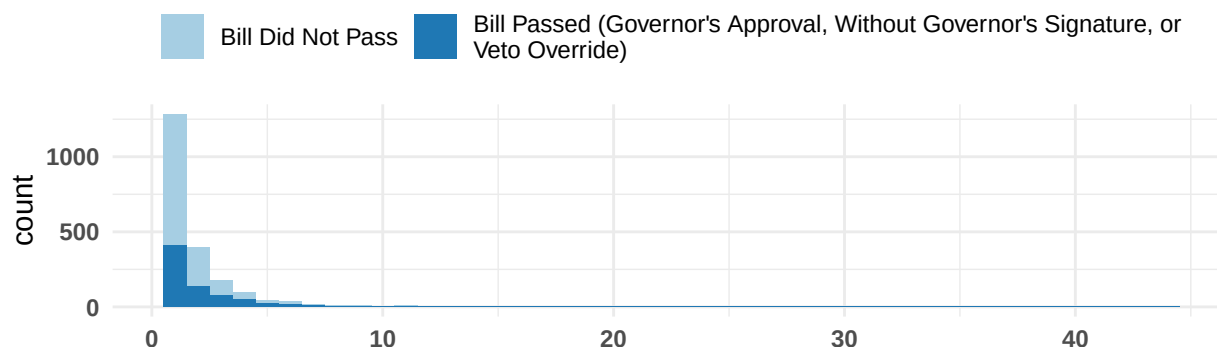


Figure 5: Pass Rate Distribution of the Number of Names Added to a Bill

Note. Excludes bills with zero names added.

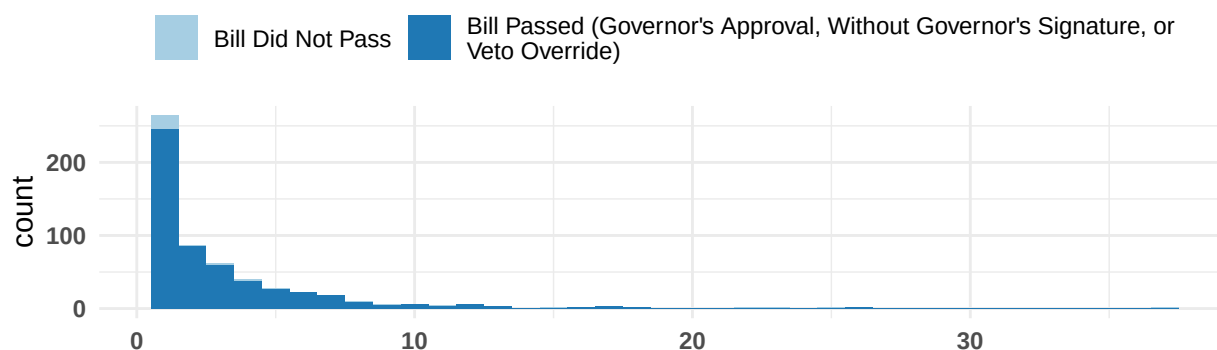


Figure 6: Pass Rate Distribution of Number of Bills Amended into Package Bills

Note. Excludes bills that are not package bills.

Training and Testing Data Sets

Due to sample size, 114 bills that were withdrawn were excluded from the final data set. Based on exploratory data analysis, 11 predictors were selected for inclusion in the final training models. These predictors were selected because they were not overly predictive of the outcome variable. Inclusion of highly correlated predictors can potentially lead to overfitting, decreasing the model's ability to correctly predict new data. The variables selected for model testing are shown in Table 4.

To ensure that no data leakage occurs during the machine learning training phase, the final data set was partitioned into a main data set that included the 100th through 107th legislative sessions ($n = 9,883$) and a final holdout data set that included the 108th legislative session ($n = 1,587$).

```
leg_final <- leg_labels |>
  filter(Bill.Withdrawn != "Bill Withdrawn") |>
  mutate(across(where(is.character), ~factor(.))) |>
  select(Passed.All, IsCarryoverBill, Is.Term.Limit, HasABill, Placed.GenFile,
```

Table 4: Legislative Variables Selected for Model Testing

Variable	Type	Description
Passed (All Bills)	Binary outcome	If a bill or provisions of a bill became law
Legislature	Integer predictor	Legislative session (e.g., 100, 101, 102)
Carryover Bill	Binary predictor	If a bill was carried over from the first to second regular session
Term Limit	Binary predictor	If a bill was introduced by a term-limited senator
Has A Bill	Binary predictor	If a bill had a corresponding A bill
General File	Binary predictor	If a bill was placed on General File
Biennium Session	Categorical predictor	Whether bill was introduced in first regular, second regular, or special session
Introducer Type	Categorical predictor	Whether bill was introduced by a senator, speaker, or committee
Bill Package Category	Categorical predictor	Whether bill was a standalone bill, a package bill, or amended into a package bill
Priority Owner	Categorical predictor	Whether bill was a priority bill of the speaker, a committee, a senator, or not a priority bill
Committee	Categorical predictor	Which committee the bill was referred or rereferred to (18 categories)
Number of Names Added	Integer predictor	Number of names added to a bill (ranges from 0 to 44)

```

Introduced.Biennium, Introducer.Type, Bill.Package.Category,
Priority.Owner, Committee, Leg, Num.NameAdded)

leg_main <- leg_final |>
  filter(Leg != 108)

leg_final_test <- leg_final |>
  filter(Leg == 108)

```

Preprocessing

Dummy Coding

Before training the machine learning algorithms, the main data set was preprocessed to transform all predictors into dummy variables using the `caret` package. Binary variables were transformed into 0 and 1 values, whereas categorical variables were transformed into $k - 1$ dummy variables with 0 and 1 values. Dummy coding is necessary for distance-based machine learning algorithms that require numeric data, such as k nearest neighbor.

```

predictors <- select(leg_main, -Passed.All)
dummies <- dummyVars(~ ., data = predictors, fullRank = TRUE)
leg_dummy <- data.frame(predict(dummies, newdata = predictors))

leg_dummy$Passed.All <- leg_main$Passed.All
str(leg_dummy, width = 70, strict.width = "wrap")

```

```
## 'data.frame': 9883 obs. of 33 variables:
## $ IsCarryoverBill.Is.Not.Carryover.Bill : num 1 1 1 1 1 0 1 1 0 1 ...
## $ Is.Term.Limit.Was.Not.Introduced.by.Term.Limited.Senator: num 1 1 1
## 1 1 1 1 1 1 1 ...
## $ HasABill.Has.a.Corresponding.A.Bill : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Placed.GenFile.Was.Not.Placed.on.General.File : num 0 0 1 1 0 1 1 0
## 1 1 ...
## $ Introduced.Biennium.Second : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Introduced.Biennium.Special : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Introducer.Type.Senator : num 0 0 1 1 1 1 1 1 1 1 ...
## $ Introducer.Type.Speaker : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Bill.Package.Category.Package.Bill : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Bill.Package.Category.Standalone.Bill : num 1 1 1 1 1 1 1 1 1 0 ...
## $ Priority.Owner.Committee : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Priority.Owner.Senator : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Priority.Owner.Is.Not.a.Priority.Bill : num 1 1 1 1 1 1 1 1 1 1 ...
## $ Committee.Appropriations : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Banking..Commerce.and.Insurance : num 0 0 0 0 0 0 0 0 0 0
## ...
## $ Committee.Business.and.Labor : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Education : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Executive.Board : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.General.Affairs : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Government..Military.and.Veterans.Affairs : num 0 0 0 0 1
## 0 1 0 0 0 ...
## $ Committee.Health.and.Human.Services : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Judiciary : num 0 0 0 0 0 1 0 1 0 0 ...
## $ Committee.Natural.Resources : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Nebraska.Retirement.Systems : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.None : num 1 1 0 0 0 0 0 0 0 0 ...
## $ Committee.None..A.Bills. : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Redistricting : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Committee.Revenue : num 0 0 1 1 0 0 0 0 1 1 ...
## $ Committee.Transportation.and.Telecommunications : num 0 0 0 0 0 0 0
## 0 0 0 ...
## $ Committee.Urban.Affairs : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Leg : num 100 100 100 100 100 100 100 100 100 100 ...
## $ Num.NameAdded : num 0 0 0 1 0 5 0 0 1 0 ...
## $ Passed.All : Factor w/ 2 levels "No","Yes": 2 2 1 1 2 1 1 2 1 2 ...
```

Near Zero Variance

To determine whether any predictors have one or very few unique values, caret's `nearZeroVar()` function was used to identify predictors that contained highly unbalanced data. The `freqRatio` column compares the most prevalent value over the second most frequent value.

```
nzv <- nearZeroVar(leg_dummy, saveMetrics = TRUE)
nzv[,][nzv$nzv,]
```

	freqRatio	percentUnique	zeroVar	nzv
Introduced.Biennium.Special	234.30952	0.02023677	FALSE	TRUE
Introducer.Type.Senator	19.50415	0.02023677	FALSE	TRUE

## Introducer.Type.Speaker	65.32886	0.02023677	FALSE	TRUE
## Priority.Owner.Committee	21.20899	0.02023677	FALSE	TRUE
## Committee.Business.and.Labor	23.70750	0.02023677	FALSE	TRUE
## Committee.Executive.Board	52.42162	0.02023677	FALSE	TRUE
## Committee.General.Affairs	40.70042	0.02023677	FALSE	TRUE
## Committee.Natural.Resources	19.80632	0.02023677	FALSE	TRUE
## Committee.Nebraska.Retirement.Systems	65.77703	0.02023677	FALSE	TRUE
## Committee.None	1234.37500	0.02023677	FALSE	TRUE
## Committee.Redistricting	448.22727	0.02023677	FALSE	TRUE
## Committee.Urban.Affairs	26.83944	0.02023677	FALSE	TRUE

Recoding Infrequent Subcategories

Based upon this analysis, bills introduced during the special session, as well as bills referred to certain committees have near zero variance. Bills without a committee assignment are only introduced during a special session. Similarly, the Redistricting Committee is not a standing committee and typically only convenes during special sessions. Therefore, special session bills, bills referred to the Redistricting Committee, and bills without a committee assignment were excluded.

To avoid excluding other bills from analysis, the Committee predictor was recategorized such that committees with fewer bills were combined into one level “Other”. Similarly, Introducer Type was recategorized into two levels: “Senator” and “Committee or Speaker”. Bill Package was categorized into two levels: “Standalone Bill” or “Package Bill or Provisions Amended”. Priority Owner was recategorized into two levels: “Is a Priority Bill” or “Not a Priority Bill”.

```
leg_main_final <- leg_main |>
  filter(Introduced.Biennium != "Special",
         Committee != "Redistricting",
         Committee != "None") |>
  mutate(Cmte = factor(case_when(Committee == "Business and Labor" ~ "Other",
                                   Committee == "Executive Board" ~ "Other",
                                   Committee == "General Affairs" ~ "Other",
                                   Committee == "Natural Resources" ~ "Other",
                                   Committee == "Nebraska Retirement Systems" ~ "Other",
                                   Committee == "Urban Affairs" ~ "Other",
                                   TRUE ~ Committee)),
         Introducer = factor(if_else(
           Introducer.Type == "Senator", "Senator", "Committee or Speaker")),
         Is.Standalone = factor(if_else(Bill.Package.Category == "Standalone Bill",
                                         ~ "Standalone Bill", "Package Bill or Provisions Amended")),
         Is.Priority = factor(if_else(Priority.Owner == "Is Not a Priority Bill", "Not a
                                         ~ Priority Bill", "Is a Priority Bill")))|>
  select(-c(Committee, Introducer.Type, Bill.Package.Category, Priority.Owner)) |>
  mutate(across(where(is.factor), ~fct_drop()))
```

Dummy Coding following Subcategory Recode

The preprocessing procedure for creating a complete set of dummy variables was performed on the final main data set. In the data, *Y* is *Passed.All*, which is a factor of 1 (bill passed) and 2 (bill did not pass). *Passed.All* includes all bills that passed, including rider bills that were amended into package bills, as well as standalone bills and package bills that passed.

```

predictors <- select(leg_main_final, -Passed.All)
dummies <- dummyVars(~ ., data = predictors, fullRank = TRUE)
leg_dummy <- data.frame(predict(dummies, newdata = predictors))

leg_dummy$Passed.All <- leg_main_final$Passed.All

str(leg_dummy, width = 70, strict.width = "wrap")

## 'data.frame': 9829 obs. of 21 variables:
## $ IsCarryoverBill.Is.Not.Carryover.Bill : num 1 1 1 0 1 1 0 1 1 1 ...
## $ Is.Term.Limit.Was.Not.Introduced.by.Term.Limited.Senator: num 1 1 1
## 1 1 1 1 1 1 ...
## $ HasABill.Has.a.Corresponding.A.Bill : num 0 0 0 0 0 0 0 0 0 1 ...
## $ Placed.GenFile.Was.Not.Placed.on.General.File : num 1 1 0 1 1 0 1 1
## 0 0 ...
## $ Introduced.Biennium.Second : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Leg : num 100 100 100 100 100 100 100 100 100 100 ...
## $ Num.NameAdded : num 0 1 0 5 0 0 1 0 0 3 ...
## $ Cmte.Appropriations : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Cmte.Banking..Commerce.and.Insurance : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Cmte.Education : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Cmte.Government..Military.and.Veterans.Affairs : num 0 0 1 0 1 0 0
## 0 0 0 ...
## $ Cmte.Health.and.Human.Services : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Cmte.Judiciary : num 0 0 0 1 0 1 0 0 0 0 ...
## $ Cmte.None..A.Bills. : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Cmte.Other : num 0 0 0 0 0 0 0 0 1 0 ...
## $ Cmte.Revenue : num 1 1 0 0 0 0 1 1 0 0 ...
## $ Cmte.Transportation.and.Telecommunications : num 0 0 0 0 0 0 0 0 0 0
## 0 ...
## $ Introducer.Senator : num 1 1 1 1 1 1 1 1 1 1 ...
## $ Is.Standalone.Standalone.Bill : num 1 1 1 1 1 1 1 0 1 1 ...
## $ Is.Priority.Not.a.Priority.Bill : num 1 1 1 1 1 1 1 1 1 1 ...
## $ Passed.All : Factor w/ 2 levels "No","Yes": 1 1 2 1 1 2 1 2 2 2 ...

```

Near-Zero Variance

After recoding existing predictors and removing special session predictors, only Introducer continued to show near zero variance levels, such that senator bills were 20 times more frequent than bills introduced by the Speaker or a committee.

```

nzv <- nearZeroVar(leg_dummy, saveMetrics = TRUE)
nzv[,][nzv$nzv,]

```

```

##               freqRatio percentUnique zeroVar  nzv
## Introducer.Senator 20.41394    0.02034795  FALSE TRUE

```

High Correlations

To identify whether any of the predictors were highly correlated with each other, a correlation matrix was constructed. Using caret's `findCorrelation()` function, predictors that were strongly correlated ($r = 0.75$) were flagged. Based upon this cutoff, no predictors were highly correlated with each other.

```
data_cor <- cor(leg_dummy |>
  mutate(Passed.All = if_else(Passed.All == "Yes", 1, 0))
)
findCorrelation(data_cor, cutoff = 0.75)
```

```
## integer(0)
```

Linear Combinations

Similar to the `findCorrelation()` function, caret's `findLinearCombos()` function identifies sets of linear combinations among all variables and returns a list of these combinations. For each combination, the function will incrementally remove them from the data to see if the dependencies have been resolved. As shown below, no problematic combinations were identified and no predictors were removed.

```
comboInfo <- findLinearCombos(leg_dummy |>
  mutate(Passed.All = if_else(Passed.All == "Yes", 1, 0))
)
```

```
comboInfo
```

```
## $linearCombos
## list()
##
## $remove
## NULL
```

Final Preprocessed Data Set

In the final preprocessing step, the data set was transformed on the dummy-coded main data set using caret's `preProcess()` function, which estimates the requirements to preprocess a data set according to the methods specified. Then, the `predict()` function is used to preprocess the data according to the preprocess requirements.

In order to remove the near-zero variance predictor (i.e., Introducer), as well as scale and center the two numeric variables (legislative session and number of names added), all binary dummy-coded variables were transformed into factors so that these variables would be ignored by the `preProcess()` function.

```
leg_dummy <- leg_dummy |>
  mutate(across(1:5, ~factor(.)),
    across(8:20, ~factor(.)))

leg_preproc <- preProcess(leg_dummy, method = c("scale", "center", "nzv"))
leg_preproc
```

```
## Created from 9829 samples and 21 variables
##
## Pre-processing:
## - centered (2)
## - ignored (19)
## - scaled (2)
```

```
leg_transf <- predict(leg_preproc, leg_dummy)
```

Training

The final transformed main data set was partitioned into 75% training data ($n = 7,372$) and 25% of the data was reserved for testing ($n = 2,457$). This partition was used because cross-validation resampling techniques require sufficient sample size for each resample (fold). If a 10-fold resample is used during training on the training data set, then at least 700 cases would be used for each fold.

```
y <- leg_transf$Passed.All  
  
set.seed(2013)  
index <- createDataPartition(y, times = 1, p = 0.75, list = FALSE)  
train_set <- leg_transf[index,]  
test_set <- leg_transf[-index,]
```

Resampling Method

To control the resampling techniques, caret's `trainControl()` function was used to assign the training model a repeated cross-validation method with three repeats to avoid lengthy computation time. Because the outcome variable is binary, class probabilities were computed in each resample, as well as two-class performance metric summary across all resamples. The argument `number` is set to the default value 10, which is the number of folds used in cross validation. As mentioned earlier, this number provides sufficient sample size for each subsample.

```
train_control <- trainControl(method = "repeatedcv",  
                              repeats = 3,  
                              classProbs = TRUE,  
                              summaryFunction = twoClassSummary)
```

Testing Multiple Models

In selecting various algorithms to use for training, the following algorithms were chosen for comparison:

- Generalized linear model (glm)
- Ridge regression (glmnet)
- Least absolute shrinkage and selection operator (LASSO) regression (glmnet)
- Random forest (rf)
- Decision tree (rpart)
- k -nearest neighbors (knn)

Because the outcome is binary, using a logistic regression (glm) is the most basic approach. An extension of logistic regression is either ridge regression or LASSO regression (both from the `glmnet` package), because both models use linear regression, and in the case of logistic regression, both models add a penalty term to the log likelihood function. Each models' penalty functions have different effects on the coefficients. Ridge

regression minimizes the effect of coefficients that contribute the most to the error term, whereas LASSO regression sets these coefficients to zero, thus, effectively selecting only the most relevant predictors. However, other models work well with classification data, including random forest, decision tree models, and k-nearest neighbor. These models were also included for comparison.

To compare multiple machine learning algorithms at once, caret includes a `caretList()` function that allows multiple algorithms to be passed in its `methodList` or `tuning_grids` arguments.

```
algorithms <- c("knn", "glm", "glmnet", "rf", "rpart")

lapply(algorithms, modelLookup) |> bind_rows()
```

##	model	parameter	label	forReg	forClass	probModel
## 1	knn	k	#Neighbors	TRUE	TRUE	TRUE
## 2	glm	parameter	parameter	TRUE	TRUE	TRUE
## 3	glmnet	alpha	Mixing Percentage	TRUE	TRUE	TRUE
## 4	glmnet	lambda	Regularization Parameter	TRUE	TRUE	TRUE
## 5	rf	mtry	#Randomly Selected Predictors	TRUE	TRUE	TRUE
## 6	rpart	cp	Complexity Parameter	TRUE	TRUE	TRUE

Using caret's `modelLookup()` function, GLM does not have any tuning parameters. However, `knn` requires parameter k , and `glmnet` requires parameters α (for identifying ridge or LASSO regression) and λ (for regularization). The random forest model, `rf`, will randomly select `mtry` variables from the set of predictors available, and the decision tree model, whereas `rpart` uses the complexity parameter `cp`.

Based upon these parameters, the `tuning_grids` object was created, which identifies each of the tuning parameters for the four models that require them. For k-nearest neighbor and random forest models, the tuning parameters were chosen based on the number of predictors in the entire data set ($p = 20$). For ridge and LASSO regression, alpha was set to either 0 or 1 and the regularization parameter λ was set to 10 values close to zero.

```
tuning_grids <- list(
  knn = caretModelSpec(
    method = "knn",
    tuneGrid = data.frame(k = seq(3, 21, 2))),
  glmnet = caretModelSpec(
    method = "glmnet",
    tuneGrid = expand.grid(
      alpha = 0:1,
      lambda = seq(0.0001, 0.1, length = 10))),
  rf = caretModelSpec(
    method = "rf",
    tuneGrid = data.frame(mtry = ceiling(seq(2, 19, length = 10)))),
  rpart = caretModelSpec(
    method = "rpart",
    tuneGrid = data.frame(cp = seq(0, 0.05, len = 10))
  )
)
```

Prior to running the algorithms, the seed was set and a time marker was stored. Because the `tuneList` argument contains the four models that require tuning parameters, only `glm` needed to be specified in the `methodList`. Running the algorithm training process took about 20 minutes.

```

set.seed(2013)
t1 <- Sys.time()
models <- caretList(Passed.All ~ .,
                     methodList = c("glm"),
                     data = train_set,
                     trControl = train_control,
                     tuneList = tuning_grids)
t2 <- Sys.time()

t2 - t1

```

```
## Time difference of 19.79727 mins
```

Training Model Comparison

In comparing the results of all models, caret's `resamples()` function was used to analyze the list of train objects (`models`). Because the `trainControl()` function specified a summary function for two-classes, the summary of all results includes the receiver operating characteristic (ROC) curve and its corresponding area under the curve (AUC) metrics. The summary also provides sensitivity and specificity for each model.

According to caret's documentation (Kuhn, 2019), a series of cutoffs are used to determine the outcome for classification data. From these cutoffs, the area under the ROC curve (AUC) is calculated and used to measure variable importance. AUC values range from 0.50 to 1.0, with a value of 0.50 indicating that the model is no better than chance at predicting whether a bill passes and a value of 1.0 indicates perfect discrimination between bills that passed and bills that did not pass.

```

all_models_results <- resamples(models)
summary(all_models_results)

```

```

##
## Call:
## summary.resamples(object = all_models_results)
##
## Models: knn, glmnet, rf, rpart, glm
## Number of resamples: 30
##
## ROC
##           Min.    1st Qu.    Median    Mean    3rd Qu.    Max. NA's
## knn      0.9558591 0.9622486 0.9656582 0.9659049 0.9700533 0.9754545    0
## glmnet   0.9651102 0.9730170 0.9759404 0.9754783 0.9788566 0.9838633    0
## rf       0.9654651 0.9712616 0.9747345 0.9744774 0.9775211 0.9831440    0
## rpart    0.9683851 0.9734841 0.9750403 0.9754307 0.9776366 0.9832335    0
## glm      0.9647343 0.9727987 0.9767164 0.9754144 0.9798475 0.9832135    0
##
## Sens
##           Min.    1st Qu.    Median    Mean    3rd Qu.    Max. NA's
## knn      0.8600509 0.8833638 0.8903061 0.8945239 0.9050362 0.9312977    0
## glmnet   0.8647959 0.8852771 0.8979592 0.8968970 0.9075856 0.9285714    0
## rf       0.8775510 0.8994294 0.9109415 0.9087912 0.9158163 0.9414758    0
## rpart    0.8698980 0.8909439 0.9006368 0.9008867 0.9094988 0.9363868    0
## glm      0.8549618 0.8880407 0.8954082 0.8967337 0.9114121 0.9312977    0
##

```

```
## Spec
##           Min.    1st Qu.    Median      Mean    3rd Qu.      Max. NA's
## knn      0.8666667 0.8767252 0.8942029 0.8942029 0.9072464 0.9333333 0
## glmnet 0.9246377 0.9419026 0.9476744 0.9484583 0.9586641 0.9709302 0
## rf      0.8956522 0.9137681 0.9304348 0.9291282 0.9411780 0.9623188 0
## rpart   0.8956522 0.9108064 0.9275362 0.9263091 0.9382731 0.9594203 0
## glm     0.9188406 0.9420290 0.9449275 0.9476840 0.9565217 0.9681159 0
```

Based upon the best selected parameters from the `glmnet` model, LASSO regression was selected based upon the best tuning parameter, $\alpha = 1$.

```
models$glmnet$bestTune
```

```
##      alpha lambda
## 11      1 0.0001
```

As shown by their ROC metrics, the results of the training models suggest that all models show relatively high discrimination in detecting bills that passed vs. bills that did not pass (ROCs > 0.966). Similarly, all models were able to correctly identify the majority of passed bills (sensitivity $M_s > 89.5\%$) and the majority of bills that did not pass (specificity $M_s > 89.4\%$). To determine whether the differences in these metrics were statistically different from each other, pairwise comparisons using Bonferroni p value adjustments were conducted.

```
summary(diff(all_models_results))
```

```
##
## Call:
## summary.diff.resamples(object = diff(all_models_results))
##
## p-value adjustment: bonferroni
## Upper diagonal: estimates of the difference
## Lower diagonal: p-value for H0: difference = 0
##
## ROC
##           knn           glmnet      rf           rpart           glm
## knn              -0.00957337 -0.00857250 -0.00952577 -0.00950946
## glmnet 0.000000311910              0.00100087 0.00004760 0.00006391
## rf      0.000004359696 1              -0.00095327 -0.00093696
## rpart   0.000000133254 1              1              0.00001631
## glm     0.000000009545 1              1              1
##
## Sens
##           knn           glmnet      rf           rpart           glm
## knn              -0.0023732 -0.0142673 -0.0063628 -0.0022098
## glmnet 1.00000              -0.0118942 -0.0039897 0.0001634
## rf      0.03060 0.04730              0.0079045 0.0120575
## rpart   1.00000 1.00000 0.91384              0.0041530
## glm     1.00000 1.00000 0.02348 1.00000
##
## Spec
##           knn           glmnet      rf           rpart           glm
## knn              -0.0542554 -0.0349253              -0.0321062 -0.0534811
```

```
## glmnet 0.0000000000004089          0.0193301          0.0221492  0.0007744
## rf      0.0000003021558200 0.0002025          0.0028191 -0.0185558
## rpart   0.0000004603966036 0.0001354  1.0000000          -0.0213748
## glm     0.0000000000023713 1.0000000  0.0000775604315172 0.0001269
```

According to the output, the lower diagonals show the p values for the pairwise comparison of each model, whereas the upper diagonal shows the estimated difference between the models. According to the ROC metrics, all models were highly similar to each other with the exception of the k-nearest neighbor model **knn**, whereby it significantly underperformed relative to the other four models. In terms of sensitivity, the random factors **rf** model was significantly better than **knn**, **glm**, and **glmnet** (LASSO regression). In terms of specificity, all models were significantly different from each other, with the exception of the two highest performing models, **glm** and **glmnet**, followed by the two middle-performing models, **rf** and **rpart**.

To visualize these differences, a boxplot is generated across all three metrics. As shown below, with the exception of the **knn** model, all four models have relatively similar discrimination and sensitivity with the **glm** and LASSO regression (**glmnet**) models demonstrating higher specificity.

```
bwplot(all_models_results, scales = list(x = list(relation = "free"),
                                              y = list(relation = "free")))
```

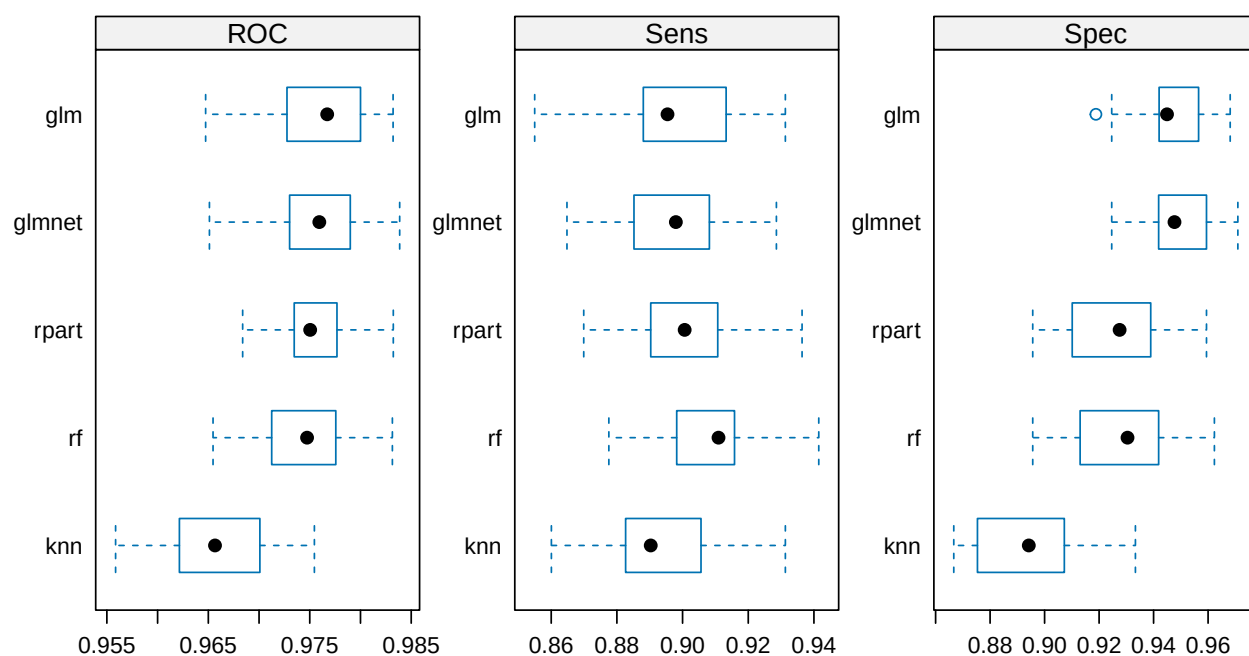


Figure 7: Boxplot of ROC, Sensitivity, and Specificity by Model

To identify which models are correlated with each other, only the **glm** model was moderately correlated with **knn** ($r = .415$) and **rf** ($r = .244$). All other models showed small correlations with each other, $r < |.300|$.

```
modelCor(all_models_results)
splom(all_models_results)
```

```
##          knn      glmnet      rf      rpart      glm
## knn      1.00000000  0.02489331 -0.01408514 -0.07030963  0.41507547
## glmnet  0.02489331  1.00000000 -0.10884337 -0.04744145 -0.02751659
```

```
## rf      -0.01408514 -0.10884337  1.00000000  0.03533229  0.24445722
## rpart   -0.07030963 -0.04744145  0.03533229  1.00000000  0.03076484
## glm      0.41507547 -0.02751659  0.24445722  0.03076484  1.00000000
```

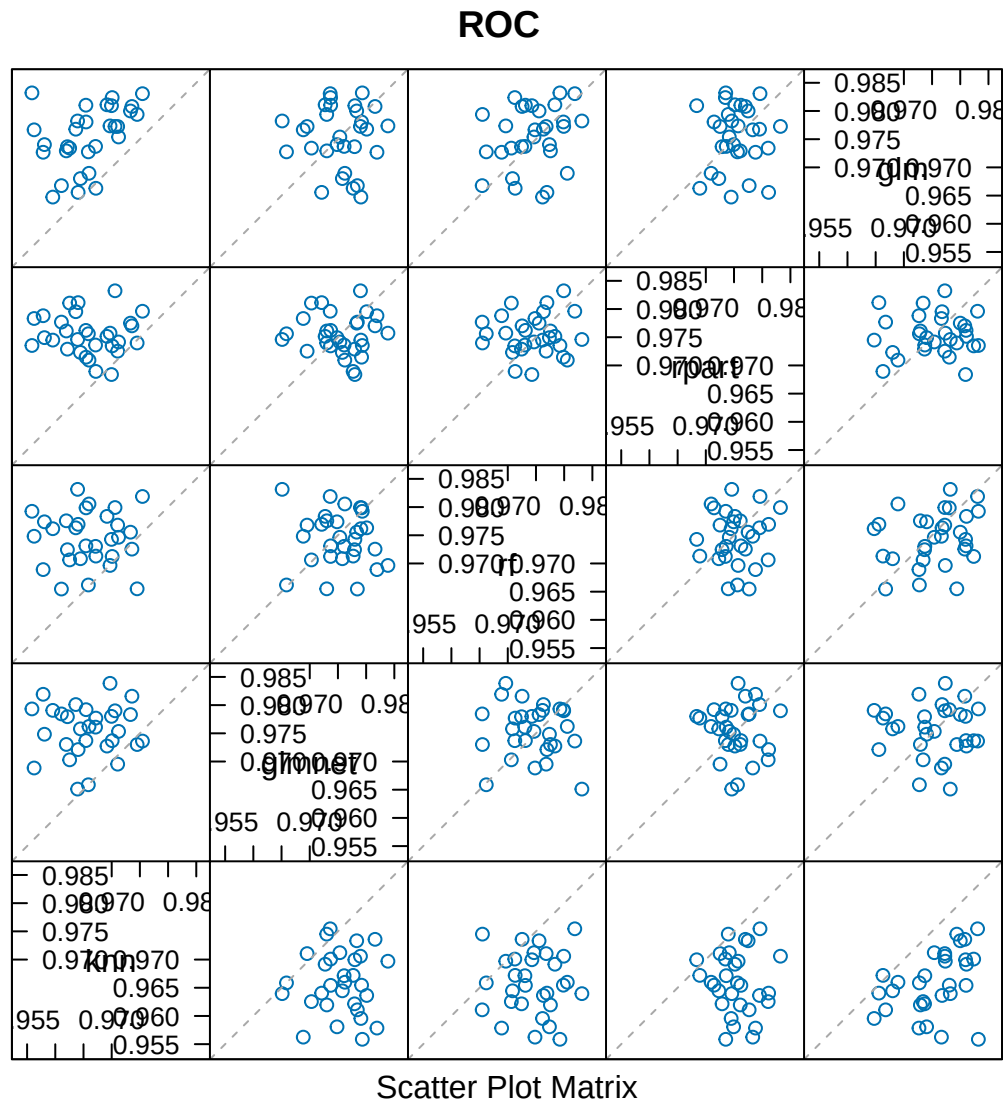


Figure 8: Scatterplot of ROC Values by Model

Evaluation on Test Set

Confusion Matrices

To test the models against the test data set, the `predict()` function is applied to the list of models to obtain the actual "Yes" or "No" values in `y_hats` and then calculate the confusion matrices for each model. From each confusion matrix, accuracy, sensitivity, and specificity were calculated.

```
y_hats <- lapply(models, function(x) predict(x, test_set))
```

```

cmats <- lapply(y_hats, function(x) {
  confusionMatrix(x, test_set$Passed.All, positive = "Yes")
})

cmats_accuracy <- sapply(cmats, function(x) x$overall["Accuracy"])
cmats_sensitivity <- sapply(cmats, function(x) x$byClass["Sensitivity"])
cmats_specificity <- sapply(cmats, function(x) x$byClass["Specificity"])

tibble(Model = names(cmats),
        Accuracy = cmats_accuracy,
        Sensitivity = cmats_sensitivity,
        Specificity = cmats_specificity) |>
  arrange(-Accuracy)

```

```

## # A tibble: 5 x 4
##   Model Accuracy Sensitivity Specificity
##   <chr>      <dbl>      <dbl>      <dbl>
## 1 glmnet    0.926      0.939      0.914
## 2 glm       0.926      0.941      0.912
## 3 rf        0.923      0.924      0.921
## 4 rpart     0.919      0.915      0.924
## 5 knn       0.902      0.898      0.906

```

The glm model demonstrated the highest sensitivity, meaning that it correctly identified 94% of all passed bills in the test data set. As shown below, the model incorrectly categorized a total of 183 bills (7.5%) out of 2,457 bills.

```
cmats$glm$table
```

```

##           Reference
## Prediction  No  Yes
##           No 1193  68
##           Yes 115 1081

```

ROC Curves

To examine ROC curves, the `predict()` function was applied to the models object with the argument `type = "prob")$Yes` to calculate the predicted probabilities of each bill in the test set. From these probabilities, the `pROC` package was used to calculate the ROC curve. The ROC data was then used to construct a data frame in order to plot each ROC curve.

```

probs <- lapply(models, function(x) predict(x, test_set,
                                             type = "prob")$Yes)

rocs <- lapply(probs, function(x) roc(test_set$Passed.All, x))

roc_data <- bind_rows(
  lapply(names(rocs), function(model) {
    roc_obj <- rocs[[model]]

    tibble(Model = model,

```

```

      FPR = 1 - roc_obj$specificities,
      TPR = roc_obj$sensitivities,
      AUC = str_extract(auc(roc_obj),
                        "\\d+\\.\\d{3}")
    )
  })
)

roc_summary_df <- roc_data |>
  select(Model, AUC) |>
  distinct() |>
  mutate(Model_labels = paste0(Model, " AUC: ", AUC))

roc_models <- unique(roc_data$Model)
roc_labels <- roc_summary_df$Model_labels
names(roc_labels) <- roc_models

p_roc <- ggplot(roc_data,
               aes(x = FPR,
                   y = TPR,
                   color = Model)) +
  geom_line(linewidth = 1) +
  geom_abline(slope = 1, intercept = 0,
              linetype = "dashed", color = "grey") +
  scale_color_brewer(palette = "Paired",
                    labels = roc_labels) +
  labs(x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme_minimal() +
  theme(panel.grid = element_blank(),
        axis.line = element_line(color = "darkgrey"),
        axis.ticks = element_line(color = "darkgrey"))

```

Based on the graph of the ROC curves, all models with the exception of `knn` had similar ROC curve metrics and were able to discriminate between bills that passed and did not pass more than 97.7% of the time. To run model comparisons on ROC curves, the `roc.test()` function from the `pROC` package was used to compare each pair of ROC curves. The default method, the DeLong test, compares the areas under two or more correlated ROC curves. To adjust p values for multiple comparisons, the Bonferroni correction was applied.

```

pairs <- combn(roc_models, 2, simplify = FALSE)

roc_comparisons <- map_dfr(pairs, function(pair) {
  model_A <- pair[1]
  model_B <- pair[2]

  test_res <- roc.test(rocs[[model_A]], rocs[[model_B]])

  tibble(
    Model.Comparison = paste(model_A, "vs.", model_B),
    D.Statistic      = test_res$statistic,
    p.value.raw      = test_res$p.value
  )
})

```

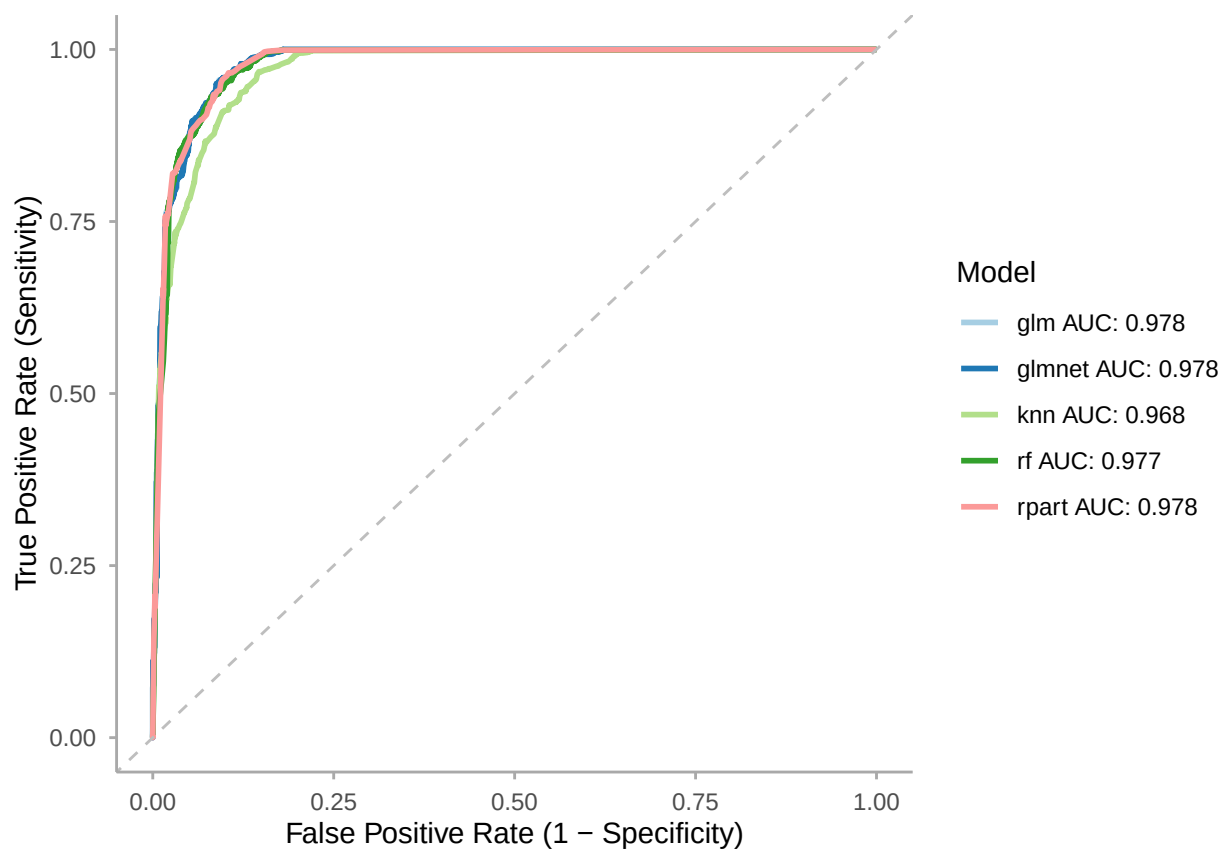


Figure 9: ROC Curves by Model


```
roc_comparisons <- roc_comparisons %>%
  mutate(p.value.adj = p.adjust(p.value.raw, method = "bonferroni"))

print(roc_comparisons)
```

```
## # A tibble: 10 x 4
##   Model.Comparison D.Statistic p.value.raw p.value.adj
##   <chr>           <dbl>         <dbl>         <dbl>
## 1 knn vs. glmnet   -4.47    0.00000791    0.0000791
## 2 knn vs. rf       -4.48    0.00000751    0.0000751
## 3 knn vs. rpart    -4.54    0.00000573    0.0000573
## 4 knn vs. glm      -4.48    0.00000735    0.0000735
## 5 glmnet vs. rf      0.698    0.485          1
## 6 glmnet vs. rpart   0.425    0.671          1
## 7 glmnet vs. glm    -0.584    0.559          1
## 8 rf vs. rpart      -0.520    0.603          1
## 9 rf vs. glm        -0.715    0.475          1
## 10 rpart vs. glm    -0.451    0.652          1
```

Based on the results, only the **knn** showed meaningful differences in discrimination relative to the other models such that the **knn** model yielded a significantly smaller AUC. Given the interpretability and simplicity of the **glm** model and the fact that this model did not significantly differ from the LASSO regression, decision tree, or random forest models, the **glm** model was chosen as the final algorithm.

Variable Importance

To identify which predictors were the most important, a singular fitted GLM model was constructed for the purpose of using caret's `varImp()` function (this function does not run on `models$glm`).

```
# Fitted GLM model
fit_glm <- train(Passed.All ~ ., method = "glm", data = train_set, trControl =
  ↪ train_control, metric = "ROC")

# Predicted values from GLM model
y_hat_glm <- predict(fit_glm, test_set)
probs_glm <- predict(fit_glm, test_set, type = "prob")$Yes

# varImp(models$glm) returns an error
plot(varImp(fit_glm))
```

To see how the top four predictors influenced the final GLM model, predicted probabilities were averaged and plotted for the predictors: Standalone, Biennium, Carryover, and General File.

```
df_plot <- data.frame(prob = probs_glm,
  fit = as.numeric(y_hat_glm),
  standalone = test_set$Is.Standalone.Standalone.Bill,
  genfile = test_set$Placed.GenFile.Was.Not.Placed.on.General.File,
  carryover = test_set$IsCarryoverBill.Is.Not.Carryover.Bill,
  biennium = test_set$Introduced.Biennium.Second) |>
  mutate(biennium = if_else(biennium == 1, "Second Regular Session", "First Regular
  ↪ Session"),
```

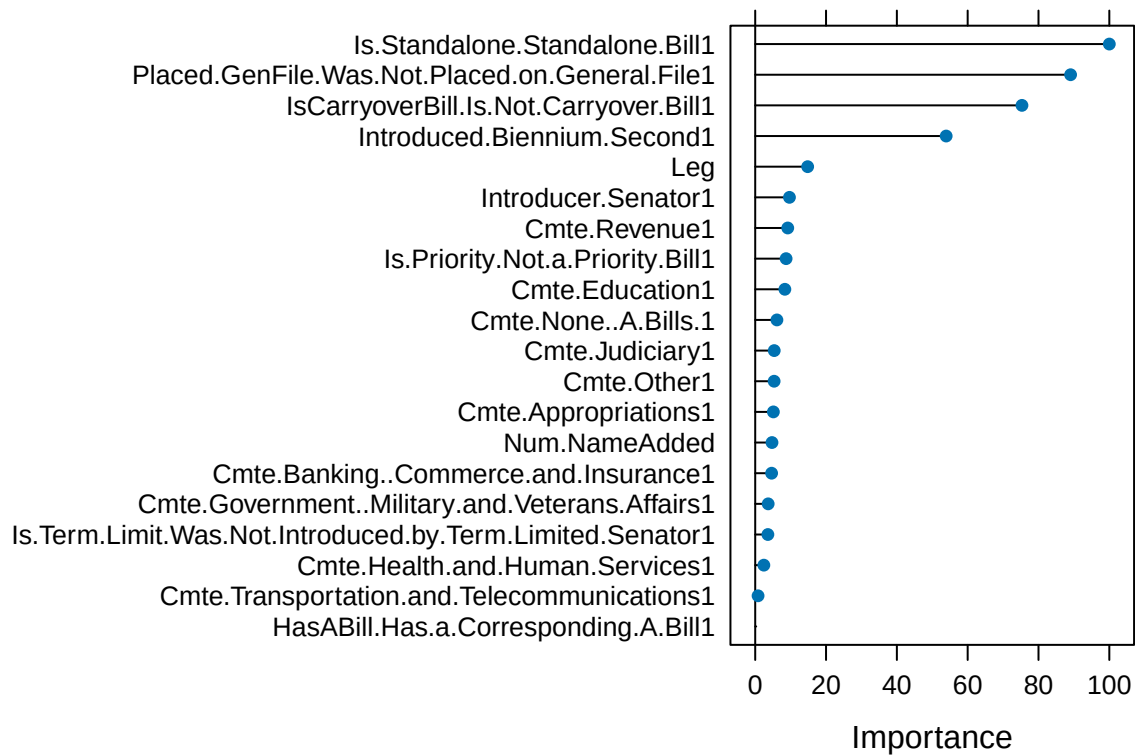


Figure 10: Variable Importance for GLM Model

```

carryover = if_else(carryover == 1, "Not a Carryover Bill", "Carryover Bill"),
genfile = if_else(genfile == 1, "Not Placed on General File", "Placed on General
↪ File"),
standalone = if_else(standalone == 1, "Is Standalone Bill", "Is Package or Rider
↪ Bill"),
) |>
pivot_longer(cols = c(standalone, carryover, genfile, biennium)) |>
group_by(name, value) |>
reframe(mean_prob = mean(prob))

df_plot |>
ggplot(aes(x = value, y = mean_prob, group = name)) +
geom_line() +
geom_point() +
geom_text_repel(aes(label = round(mean_prob, 3)),
direction = "y") +
facet_wrap(vars(name), scales = "free_x") +
labs(x = NULL, y = NULL,
title = "Mean Predicted Probabilities") +
theme(plot.title.position = "plot")

```

As shown above, being a package or rider bill is the strongest predictor of a bill passing and changes the average predicted probability of passing from 34.0% to 97.2%. Similarly, placing a bill on General File increases the average predicted probability from 14.4% to 79.3%. Introducing a bill in the first regular session and not being a carryover bill improves a bill's passage some, but not as much as being a package bill or placing a bill on General File.

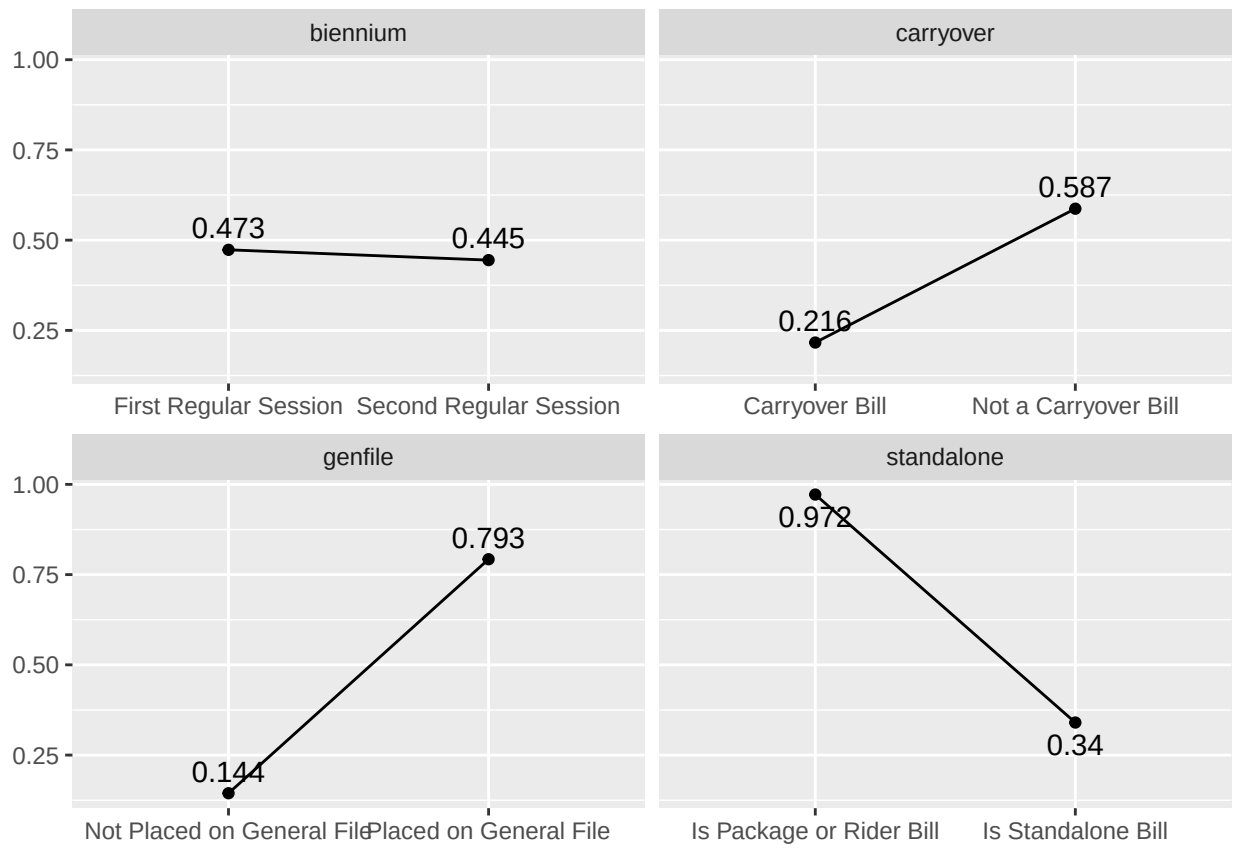


Figure 11: Mean Predicted Probabilities for Highly Important Variables in GLM Model

Results

Final Holdout Preprocessing

The final holdout data set included all introduced bills from the 108th Legislature. Before determining the performance of the GLM model on the final holdout data set, the same recoding and preprocessing procedures were conducted.

First, `leg_final_test` excluded special session bills and recoded other low-frequency subcategories into larger subcategories.

```
leg_final_test_selectvars <- leg_final_test |>
  filter(Introduced.Biennium != "Special",
         Committee != "Redistricting",
         Committee != "None") |>
  mutate(Cmte = factor(case_when(Committee == "Business and Labor" ~ "Other",
                                Committee == "Executive Board" ~ "Other",
                                Committee == "General Affairs" ~ "Other",
                                Committee == "Natural Resources" ~ "Other",
                                Committee == "Nebraska Retirement Systems" ~ "Other",
                                Committee == "Urban Affairs" ~ "Other",
                                TRUE ~ Committee)),
         Introducer = factor(if_else(
           Introducer.Type == "Senator", "Senator", "Committee or Speaker")),
         Is.Standalone = factor(if_else(Bill.Package.Category == "Standalone Bill",
           ~ "Standalone Bill", "Package Bill or Provisions Amended")),
         Is.Priority = factor(if_else(Priority.Owner == "Is Not a Priority Bill", "Not a
           ~ Priority Bill", "Is a Priority Bill")))|>
  select(-c(Committee, Introducer.Type, Bill.Package.Category, Priority.Owner)) |>
  mutate(across(where(is.factor), ~fct_drop(.)))
```

Then, the final holdout test data set was preprocessed to dummy code categorical variables and center and scale continuous variables. Because all bills in this data set were from the same legislative session, the method `nzv` (non-zero variance) was removed from the method argument in the `preProcess()` function to prevent the caret package from removing `Leg` as a predictor.

```
predictors_test <- select(leg_final_test_selectvars, -Passed.All)
dummies_test <- dummyVars(~ ., data = predictors_test, fullRank = TRUE)
leg_dummy_test <- data.frame(predict(dummies_test, newdata = predictors_test)) |>
  mutate(across(1:5, ~factor(.)),
         across(8:20, ~factor(.)))
leg_dummy_test$Passed.All <- leg_final_test_selectvars$Passed.All

# Took "nzv" off because Leg = 108
leg_preproc_test <- preProcess(leg_dummy_test, method = c("scale", "center"))
```

```
## Warning in preProcess.default(leg_dummy_test, method = c("scale", "center")):
## These variables have zero variances: Leg
```

```
leg_preproc
```

```
## Created from 9829 samples and 21 variables
```

```
##
## Pre-processing:
##   - centered (2)
##   - ignored (19)
##   - scaled (2)
```

```
leg_transf_test <- predict(leg_preproc_test, leg_dummy_test)
```

Final Model Performance

The final testing showed that the GLM algorithm was highly successful at discriminating between passed bills and bills that did not pass (AUC = 0.973) with sensitivity percentages above 94.3%. The model showed less success in correctly predicting bills that did not pass (specificity = 87.3%). As shown by the confusion matrix, the GLM model correctly identified 90.43% of all bills ($n = 1,504$). These metrics were slightly lower than the corresponding test set data ($n = 2,457$), which was a subset of the main training set (AUC = 0.978, sensitivity = 0.941, specificity = 0.912)

```
y_hat_glm_final <- predict(fit_glm, leg_transf_test)
probs_glm_final <- predict(fit_glm, leg_transf_test, type = "prob")$Yes

confusionMatrix(y_hat_glm_final, leg_transf_test$Passed.All, positive = "Yes")

roc_glm_final <- roc(leg_transf_test$Passed.All, probs_glm_final)

plot(roc_glm_final, print.auc = TRUE)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No  Yes
##           No  728  38
##           Yes 106 632
##
##           Accuracy : 0.9043
##           95% CI : (0.8883, 0.9187)
##           No Information Rate : 0.5545
##           P-Value [Acc > NIR] : < 0.00000000000000022
##
##           Kappa : 0.8081
##
## Mcnemar's Test P-Value : 0.0000000236
##
##           Sensitivity : 0.9433
##           Specificity : 0.8729
##           Pos Pred Value : 0.8564
##           Neg Pred Value : 0.9504
##           Prevalence : 0.4455
##           Detection Rate : 0.4202
##           Detection Prevalence : 0.4907
##           Balanced Accuracy : 0.9081
##
##           'Positive' Class : Yes
##
```

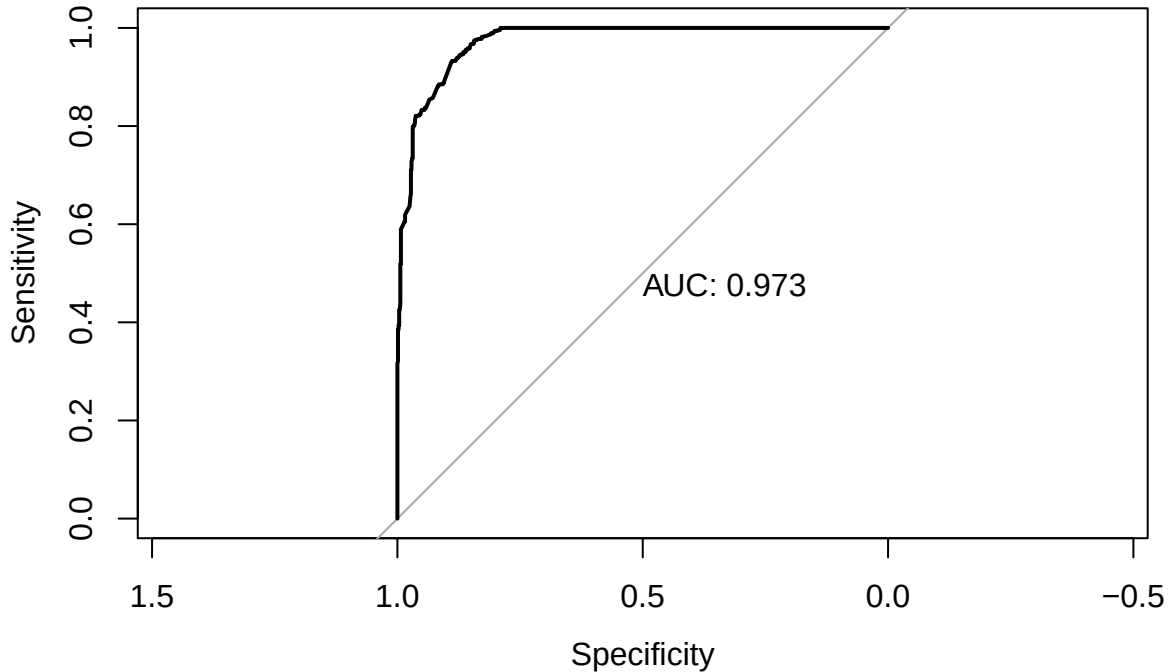


Figure 12: ROC Curve in Final GLM Model Predicting Passed Legislative Bills in 108th Nebraska Legislature

Discussion

Overview

The purpose of this project was to evaluate the effectiveness of machine learning algorithms on predicting the passage of Nebraska legislative bills. To do this, 11,584 legislative bills introduced in the Nebraska Legislature were scraped from journal chronologies spanning 2007 to 2024. Preliminary data analysis revealed that once a bill advances from the first round of debate (i.e., General File), it is highly likely to pass. Thus, bill characteristics relating to second and third rounds of debate were excluded from the final data set. Other bill characteristics that were included as predictors in the model included, but were not limited to, whether or not the bill was a standalone bill, package bill, or “rider” bill, whether or not the bill was a carryover bill from the first regular session of the biennium, and referred committee. The outcome variable (passage of a bill into law) was expanded to include provisions of legislative bills that were amended into package bills.

Using a training data set of 7,372 legislative bills, model evaluation was conducted via repeated 10-fold cross-validation across three complete iterations for five different models: generalized linear model, LASSO regression, random forest, decision tree, and k-nearest neighbor. Based on mean area under the ROC curve values, as well as sensitivity and specificity percentages, the generalized linear model and LASSO regression demonstrated the best overall predictive accuracy. However, given that there were no significant differences between the two models, the generalized linear model was chosen to predict the accuracy of an independent set of legislative bills ($N = 1,587$). The final generalized linear model correctly predicted bill outcomes for 90.4% of bills with an exceptionally high discriminative ability ($AUC = 0.973$).

Potential Impact

This work has the potential to be very useful to Nebraska policymakers, advocates, and communities that need to make decisions regarding the success rate of bills passed in the Nebraska Legislature. Given that

package bills had the highest chance of success, lawmakers may want to think strategically about bills that are amended into package bills. This work also suggests that the trend of using package bills as vehicles for bill passage may be indicative of structural changes within the Nebraska Legislature. Lawmakers may also want to use this work to prioritize their legislative priorities within a given legislative session. Standalone bills, bills not yet placed on General File, and carryover bills had much lower predictive probabilities of passing. If a standalone bill does not leave a committee and is carried over to the second regular session, lawmakers may want to find a package bill as a vehicle for passage. If the bill is highly consequential, knowing the chances of success before attempting to pass the bill as a standalone bill may help with decision making when assigning time and resources toward passing the bill.

Limitations

Although this work may be highly impactful, this research project is not without limitations. First, there were many important bill characteristics that were not readily accessible from the journal chronologies, including the size of the fiscal impact. Each legislative bill that requires an appropriation, will have a corresponding A-bill. However, the size of the fiscal impact was not included in this analysis. Second, data collection process did not distinguish between bills that varied between certain categories at certain times during the legislative session. For example, there may have been bills that were initially amended into a package bill but then failed to advance as a package bill and returned to its standalone state. These bills would be categorized as standalone bills, but may also warrant its own category for bill type. Finally, this project did not track either the political affiliations or legislative district of the bill introducer, which would likely alter the likelihood of a bill passing. Although the Nebraska unicameral is officially nonpartisan, the political realities likely influence the lawmaking process, which was not captured in this data set.

Future Work

In the future, policy researchers should continue gathering journal chronologies for current and future legislative sessions. As more training data is added, the predictive accuracy of the model would likely improve. Policy researchers should also consider other information that could be scraped from the journal chronologies or other legislative archived documents. With emerging artificial intelligence technologies, there may be new methods of collecting information, such as fiscal impact based upon the legislative records on the Nebraska Legislature's website. In addition, including the political affiliation or legislative district may require more advanced machine learning algorithms given that there are 49 different legislative districts that change after each decennial census.

Conclusion

To evaluate the effectiveness of machine learning algorithms for predicting Nebraska legislative bill passage, this project compared five common algorithms for classification data (i.e., generalized linear model, LASSO regression, random forest, decision trees, and k-nearest neighbor). After training each algorithm on a data set of more than 7,000 legislative bills with 20 different predictors, the generalized linear model and LASSO regression achieved the highest predictive accuracy. As the simpler model, the generalized linear model was, then, used to predict more than 1,500 legislative bills of an independent data set. The results of the generalized linear model demonstrated an outstanding level of discrimination, providing a strong basis for forecasting whether a bill will become Nebraska state law.

References

- Kuhn. (2019). *The caret Package*. <https://topepo.github.io/caret/variable-importance.html#model-independent-metrics>
- Nebraska Legislature. (2025). *Chronology of bills: One hundred ninth legislature first session*. Nebraska Legislature. <https://nebraskalegislature.gov/FloorDocs/109/PDF/Journal/r1chronology.pdf>
- Nebraska Legislature. (2026a). *Lawmaking in Nebraska*. Nebraska Legislature. <https://nebraskalegislature.gov/about/lawmaking.php>
- Nebraska Legislature. (2026b). *Worksheet: One hundred ninth legislature - second session*. Nebraska Legislature. <https://nebraskalegislature.gov/FloorDocs/109/PDF/WorkSheet/r2dayfinal.pdf>
- Nebraska Legislature. (n.d.). *Unicam focus*. Nebraska Legislature. <https://nebraskalegislature.gov/education/lesson3.php>
- Unicameral Update. (2026). Nebraska Legislature. <https://update.legislature.ne.gov/>